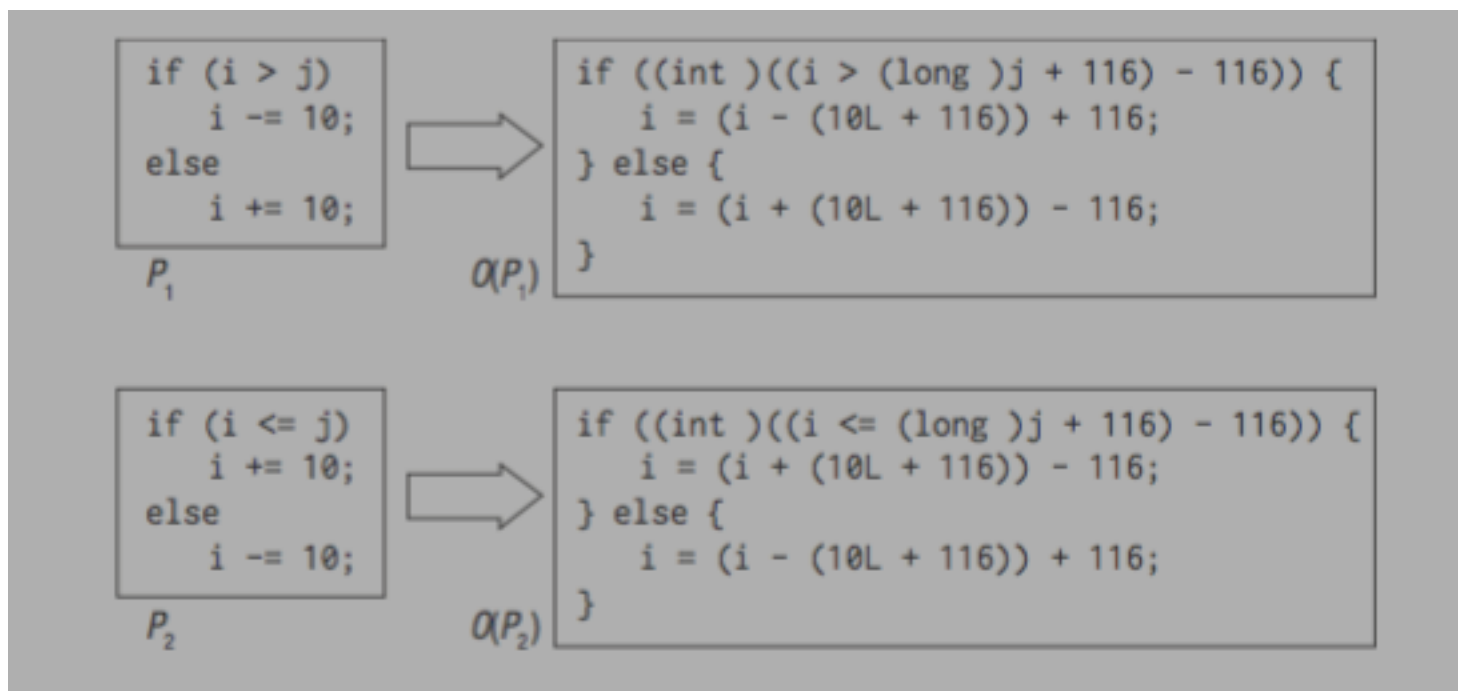


COMP3052.SEC Computer Security

Session DT: Metamorphic Security 会议安全 DT: 变形式安全

核心:即使我们不知道程序输出是否正确, 我们仍然可以检测bug



Acknowledgements

- Some of the materials come directly from previous teachers of the Computer Security module, and other sources ...
- Thank you to (amongst others):
 - Michel Valstar, Milena Radenkovic, Mike Pound, ...
 - T. Y. Chen, Fei-Ching (Diana) Kuo, George Zhou

Thanks, too, to Alex for letting me talk with you!

Motivation

What if we could know that a program is buggy, even if we could not tell whether or not its observed output is correct? This is one of the key strengths of metamorphic testing, a technique where failures are not revealed by checking an individual concrete output, but by checking the relations among the inputs and outputs of multiple executions of the program under test. Two decades after its introduction, metamorphic testing has become a fully-fledged testing technique with successful applications in multiple domains, including online search engines, autonomous machinery, compilers, Web APIs, and deep learning programs, among others.

- Sergio Segura, Dave Towey, Zhi Quan Zhou, and Tsong Yueh Chen (2018) “Metamorphic Testing: Testing the Untestable,” IEEE Software 37 (3), 46-53. [Online] Available: <https://doi.org/10.1109/MS.2018.2875968>

Goals of Testing

- What are two major possible reasons for conducting testing? 进行测试的两个主要原因是什么？
 - Testing is intended to
 - (1) **show that a program does what it is intended to do** (Validation Testing); and 证明一个程序确实能实现其预期功能（验证测试）
 - (2) **discover program defects** (Defect Testing) before it is put into use 在程序投入使用前发现其缺陷（缺陷测试）
 - Can **reveal the presence of errors** **NOT** their absence (Dijkstra, 1969) 能够发现错误的存在，而非错误的不存在

Key Testing Terminology

- xUT

- **X Under Test**: function, class, program, software ...

X 被测试对象：功能、类、程序、软件等->即被测试的软件, 可以是function,class,program, software system

- Test Case

一组输入用于一次execution

- Combination of inputs necessary for a single execution of **xUT**
用于单次执行 xUT 所需的输入组合
 - Sometimes also includes expected output
有时还包括预期输出

Oracle判断输出是否正确的方法,例如数学函数有标准答案

Oracle problem是当没有案发知道程序输出是否正确, 即No practical way to verify correctness
数据库问题

The Oracle Problem

- When testing the software – the xUT – how do we know it is behaving correctly?

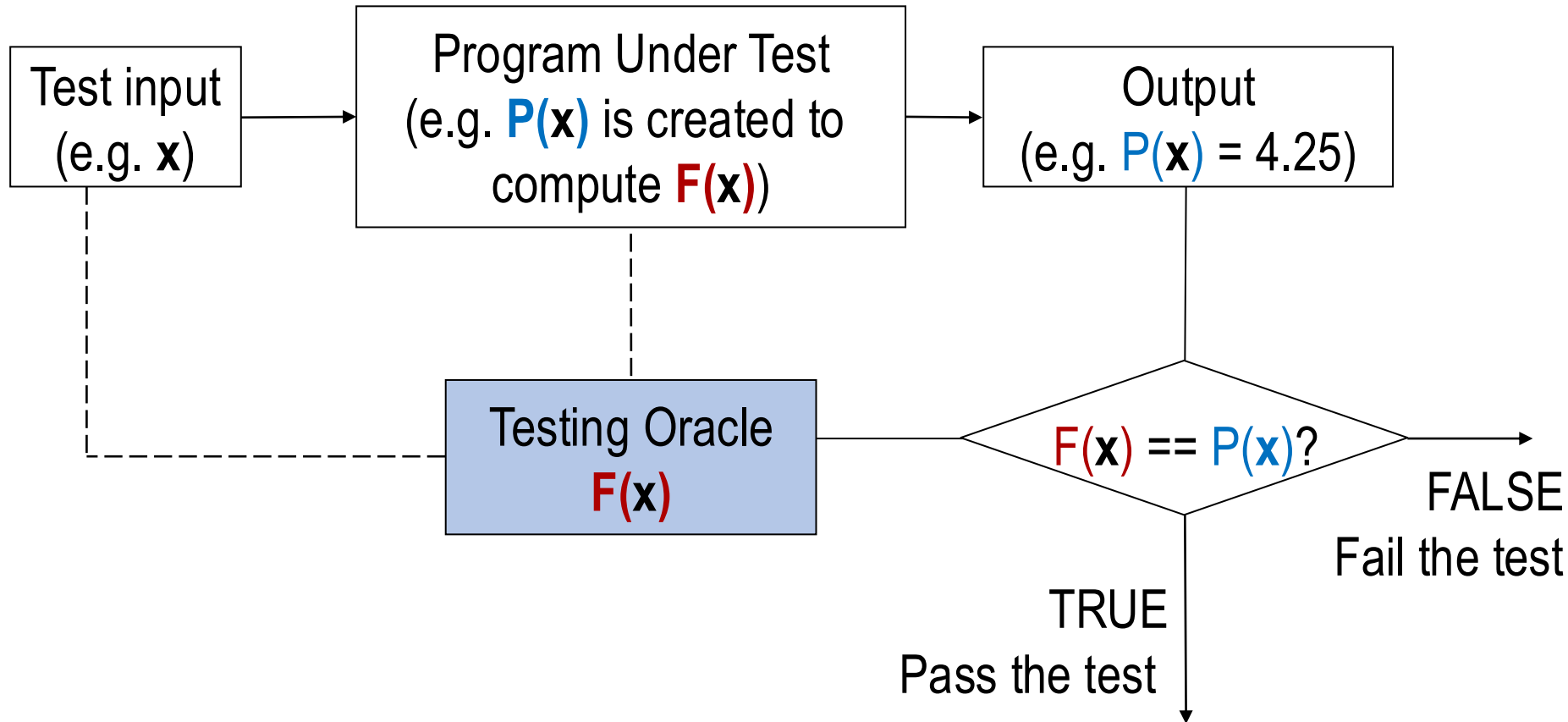
在测试软件（即 xUT）时，我们如何确认其运行状态是正常的

- An Oracle
 - Some way to know if the output from the xUT is “correct”
一种方法来判断 xUT 的输出是否“正确”
 - Maybe an older version of the software
可能是该软件的旧版本
- The Oracle Problem
 - When an oracle isn't available
当没有数据库可用时
 - ... or is available, but it isn't practical to use it
或者虽然可用，但使用起来并不实际
- Systems facing the Oracle Problem are called Untestable
面临数据库问题的系统被称为“无法测试的”

The Oracle Problem

- The Oracle Problem has a very severe impact on how software testing can be applied
数据库问题 ” 对软件测试的应用方式产生了极为严重的影响
- One of the ways that has been developed to alleviate the Oracle Problem is Metamorphic Testing (MT) 为缓解“数据库问题”，已开发出的一种方法是“变形测试”（MT）
- MT is not just a way to address the Oracle Problem, it is a new paradigm for testing!
MT 不仅仅是一种解决“数据库问题”的方法，它还是一种全新的测试范式！

When Test Oracle is Usable

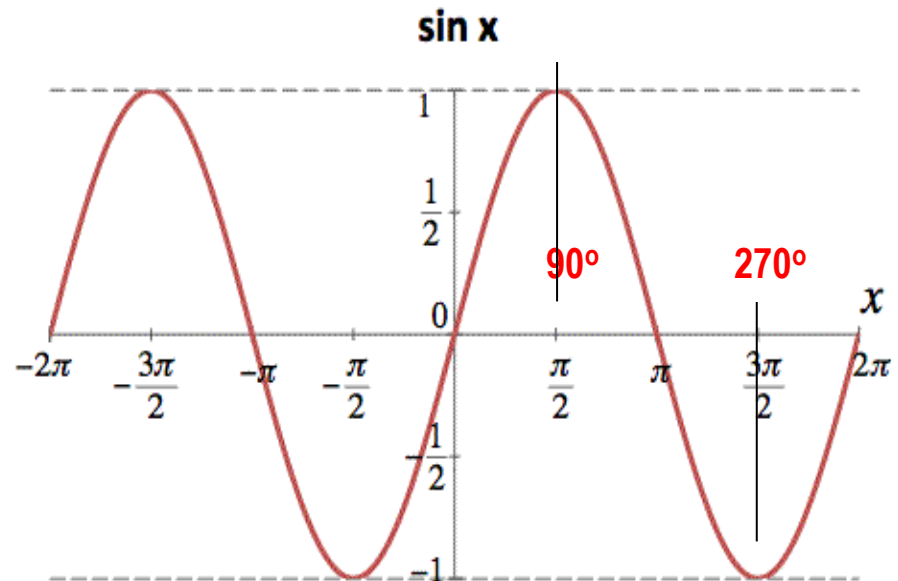


Oracle: Examples

◆ *Sine function*

- $\sin(0^\circ)=0$

- $\sin(30^\circ)=0.5$



◆ Suppose the program returns:

- $\sin(29.8^\circ)=0.5123$ is this incorrect?

- $\sin(29.8^\circ)=0.4225$ is this correct?

Oracle: Examples

◆ Consider an online hotel booking system

考虑一个在线酒店预订系统

- ◆ Suppose you wanted to find a room close to UNNC (within 2 km), costing less than RMB 200 per night, for April 10 and 11

假设您想找到一个距离宁波大学城较近（距离不超过 2 公里）、每晚费用低于 200 元人民币的房间，预订日期为 4 月 10 日和 11 日

- ◆ If the system replies that there are two available hotels (Hotel-A and Hotel-B): 如果系统回复有两家可用的酒店（酒店 A 和酒店 B）

- ◆ Can verify if they are available – phone them
可以核实它们是否可用——给他们打电话
- ◆ Probably very difficult to verify that no other eligible hotels were omitted
很难确认是否遗漏了其他符合条件的酒店

Tackling the oracle problem

- We build software based on some design to fulfil users' requirements, thus certain properties are built into the software
根据某些设计来开发软件，以满足用户的需求，因此软件中就具备了某些特定的特性。
 - These properties are the key!

Metamorphic Testing (Simplified)

- Get source (initial) test cases using some testing strategies 利用某些测试策略获取源（初始）测试用例
- Identify some properties of the problem (referred to as the metamorphic relations, MRs)
确定问题的一些特性（称为变形关系，简称 MR）
- Follow-up test cases are constructed from the source test cases with reference to the identified metamorphic relations
根据所确定的变形关系，从源测试用例中构建后续测试用例

Metamorphic Testing Example

- To test the *sine* function
- Some **properties** of the *sine* function

- $\sin(x) = \sin(360+x)$

One **source** test case, and one **follow-up**

- $\sin(x) = \sin(180-x)$

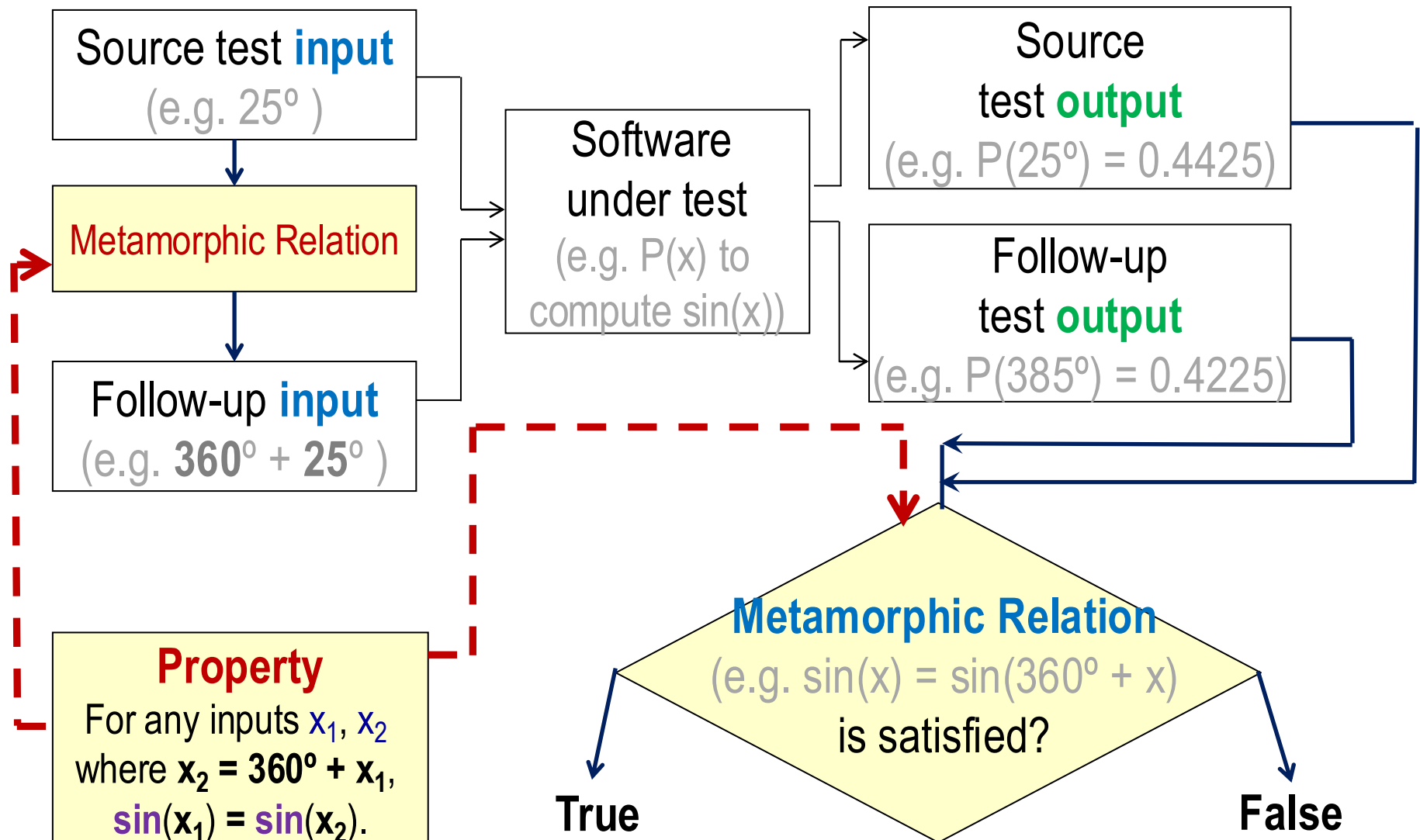
- $\sin(-x) = -\sin(x)$

Two **source** test cases, and two **follow-up**

- $\sin(3x) = 3*\sin(x) - 4*\sin^3(x)$

- $\sin^2(x) - \sin^2(y) = \sin(x+y) * \sin(x-y)$

Testing using MT (1 s; 1 f)



How to Identify MRs

- A simple approach:
 - Select an input
 - Modify it
 - Is its output somewhat predictable?
 - if yes: is any generalisation possible?

Metamorphic Testing Example

- Consider a program that reads in a series of integers and calculates their average value
- **Can you identify any MR for this software?**

Metamorphic Testing Example

- Consider a program that reads in a series of integers and calculates their average value

Changing the order of numbers should not change the output:

$$\text{AVERAGE}(\{1,4,6,7\}) == \text{AVERAGE}(\{7,4,1,6\})$$

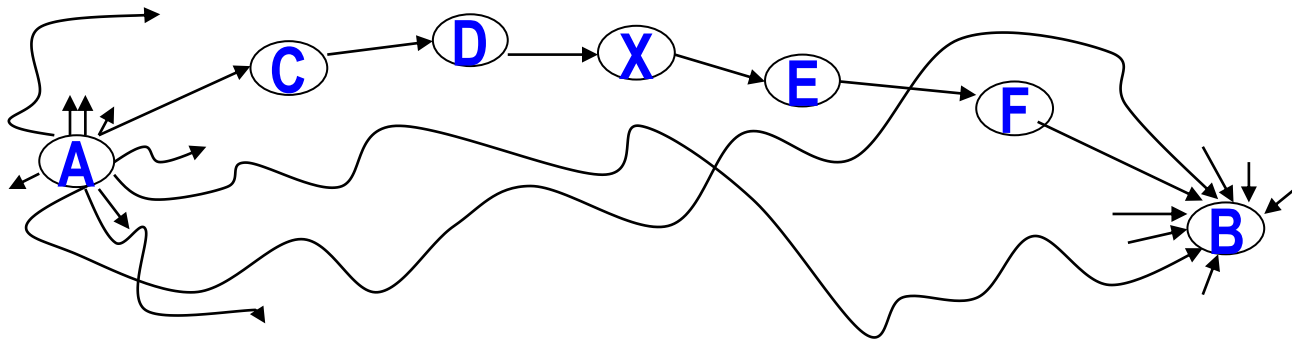
- Can you identify any MR for this software?**

MR Identification: Exercise 1

- Consider again the online hotel booking system, that you use to find a hotel room close to UNNC (within 2 km), costing less than RMB 200 per night, for April 10 and 11
- If the system replies that there are two available hotels
 - (Hotel-A and Hotel-B):
 - Can verify if they are available – phone them
 - Probably very difficult to verify that no other eligible hotels were omitted
- **Can you identify/generate MRs for this?**

MR Identification: Exercise 2

- Consider a program that **computes** the **shortest path** between two given cities, A and B
- If the program returns A->C->D->X->E->F->B, how **can we know that this is the shortest path** between Cities A and B
- Very **computationally expensive** to **verify** correctness of this single test output



- Can you identify any MR for this software?**

MR Identification: Exercise 2

Source test case

Graph G
Source A
Destination B



shortestPath(G,A,B)



{A,C,D,X,E,F,B}

Follow-up test case

Graph G
Source B
Destination A



shortestPath(G,B,A)



{B,Q,R,S,A}

$$\begin{array}{c} |\text{shortestPath}(G,A,B)| \\ = \\ |\text{shortestPath}(G,B,A)| \end{array}$$



Detecting Obfuscator Bugs



Computer, vol. 49, no. 6, pp. 48–55, 2016.

Detecting Obfuscator Bugs

- Critical security infrastructure
关键安全基础设施
- Transform an original program into an equivalent program
将原始程序转换为等效程序
- less readable 可读性降低
- to prevent attackers from analysing and understanding the original code
以防止攻击者分析和理解原始代码

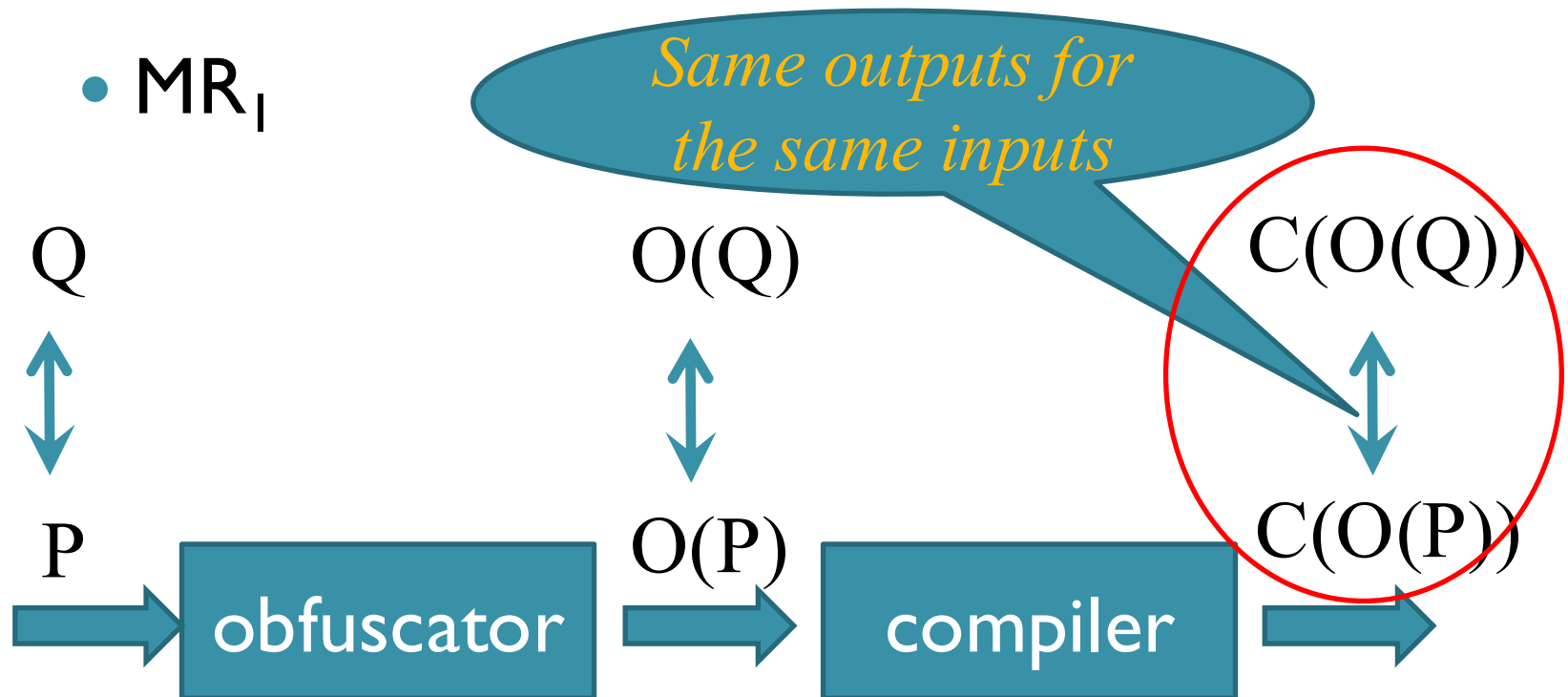
Detecting Obfuscator Bugs

- As important as compilers 与编译器同等重要
- But difficult to test (Oracle Problem)
但难以进行测试 (Oracle 问题)
- A surprising observation:
 - Compiler testing: a lot of research
编译器测试：有大量的研究
 - Obfuscator testing: almost none
代码混淆器测试：几乎没有任何研究
- Our work: the **first** to report **real bugs**
我们的成果：是首个报告实际漏洞的成果

Detecting Obfuscator Bugs

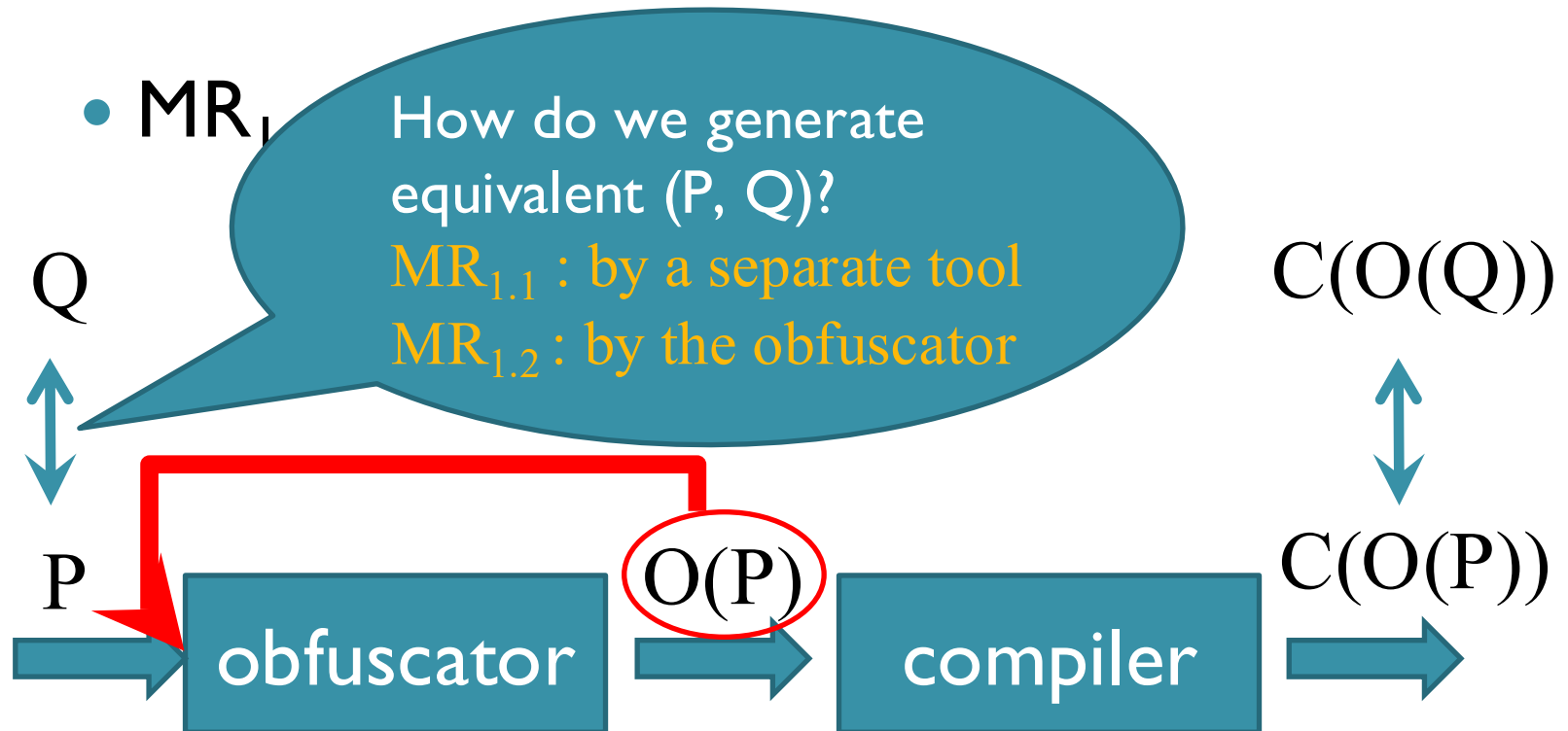
One of the metamorphic relations (MRs)

- MR_1



Detecting Obfuscator Bugs

One of the metamorphic relations (MRs)



Detecting Obfuscator Bugs

Tigress

- C obfuscator developed at the University of Arizona that supports novel defences against both static and dynamic reverse engineering

Detecting Obfuscator Bugs

Tigress

```
if (i > j)
    i -= 10;
else
    i += 10;
```

 P_1

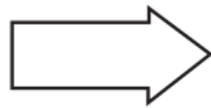
```
if (i <= j)
    i += 10;
else
    i -= 10;
```

 P_2

Detecting Obfuscator Bugs

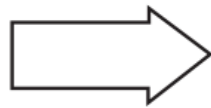
Tigress

```
if (i > j)  
    i -= 10;  
else  
    i += 10;
```

 P_1  $O(P_1)$

```
if (((int )((i > (long )j + 116) - 116))) {  
    i = (i - (10L + 116)) + 116;  
} else {  
    i = (i + (10L + 116)) - 116;  
}
```

```
if (i <= j)  
    i += 10;  
else  
    i -= 10;
```

 P_2  $O(P_2)$

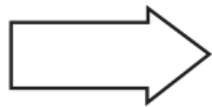
```
if ((int )((i <= (long )j + 116) - 116)) {  
    i = (i + (10L + 116)) - 116;  
} else {  
    i = (i - (10L + 116)) + 116;  
}
```

Detecting Obfuscator Bugs

Tigress

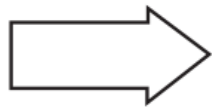
*either true or false,
i.e. 1 or 0.*

```
if (i > j)
    i -= 10;
else
    i += 10;
```

 P_1  $O(P_1)$

```
if ((int)((i > (long)j + 116) - 116)) {
    i = (i - (10L + 116)) + 116;
} else {
    i = (i + (10L + 116)) - 116;
}
```

```
if (i <= j)
    i += 10;
else
    i -= 10;
```

 P_2  $O(P_2)$

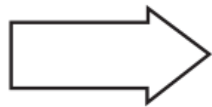
```
if ((int)((i <= (long)j + 116) - 116)) {
    i = (i + (10L + 116)) - 116;
} else {
    i = (i - (10L + 116)) + 116;
}
```

Detecting Obfuscator Bugs

Tigress

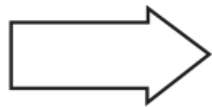
*either 1-116 or 0-116
i.e., true.*

```
if (i > j)
    i -= 10;
else
    i += 10;
```

 P_1  $O(P_1)$

```
if ((int)((i > (long)j + 116) - 116)) {
    i = (i - (10L + 116)) + 116;
} else {
    i = (i + (10L + 116)) - 116;
}
```

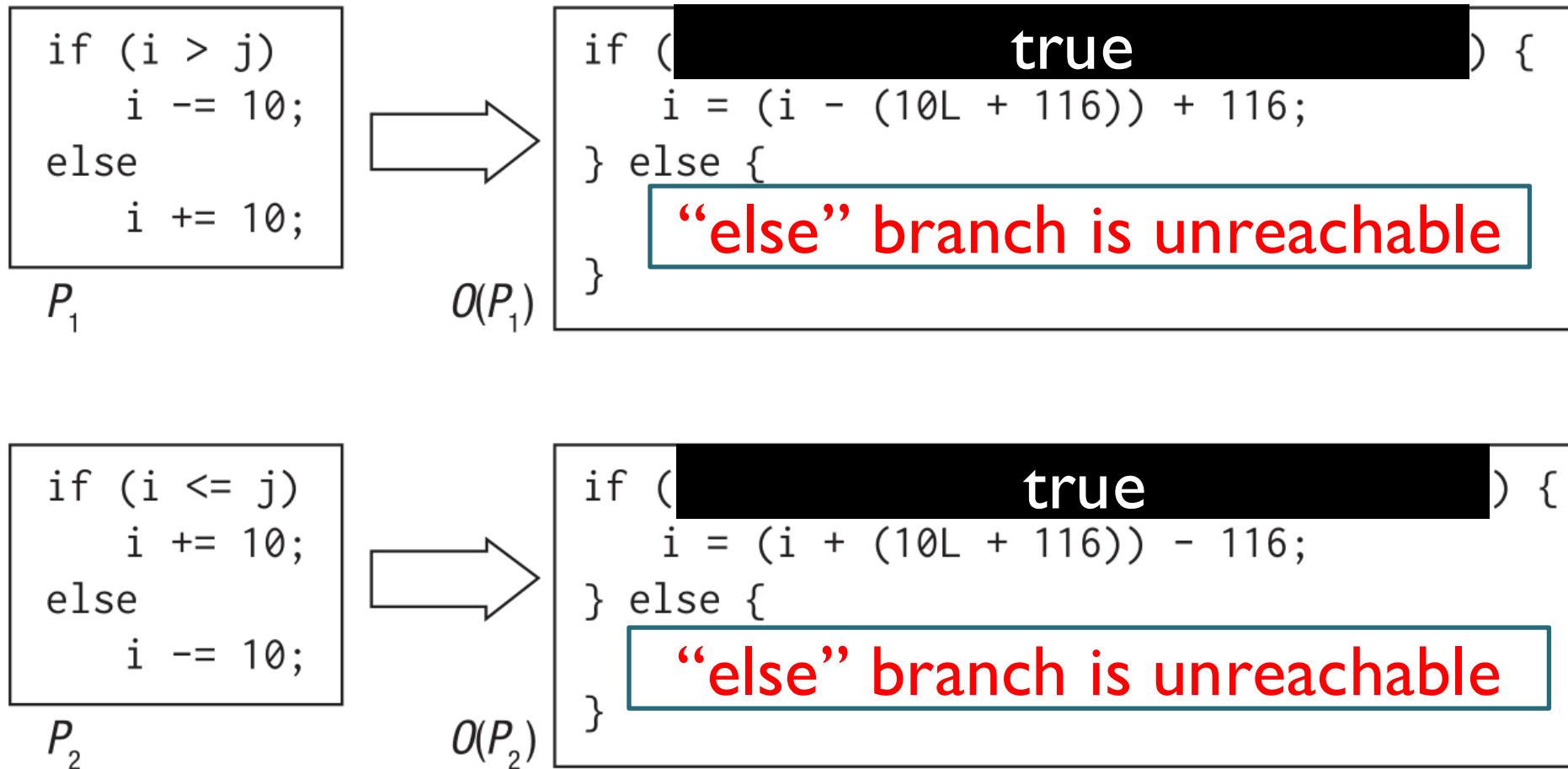
```
if (i <= j)
    i += 10;
else
    i -= 10;
```

 P_2  $O(P_2)$

```
if ((int)((i <= (long)j + 116) - 116)) {
    i = (i + (10L + 116)) - 116;
} else {
    i = (i - (10L + 116)) + 116;
}
```

Detecting Obfuscator Bugs

Tigress

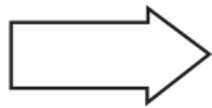


**Any MT test case (i, j) will
detect a failure because
 $i-10 \neq i+10$.**

Tigress

```
if (i > j)
    i -= 10;
else
    i += 10;
```

P_1

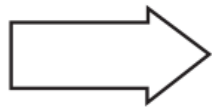


$O(P_1)$

$i = i - 10$

```
if (i <= j)
    i += 10;
else
    i -= 10;
```

P_2

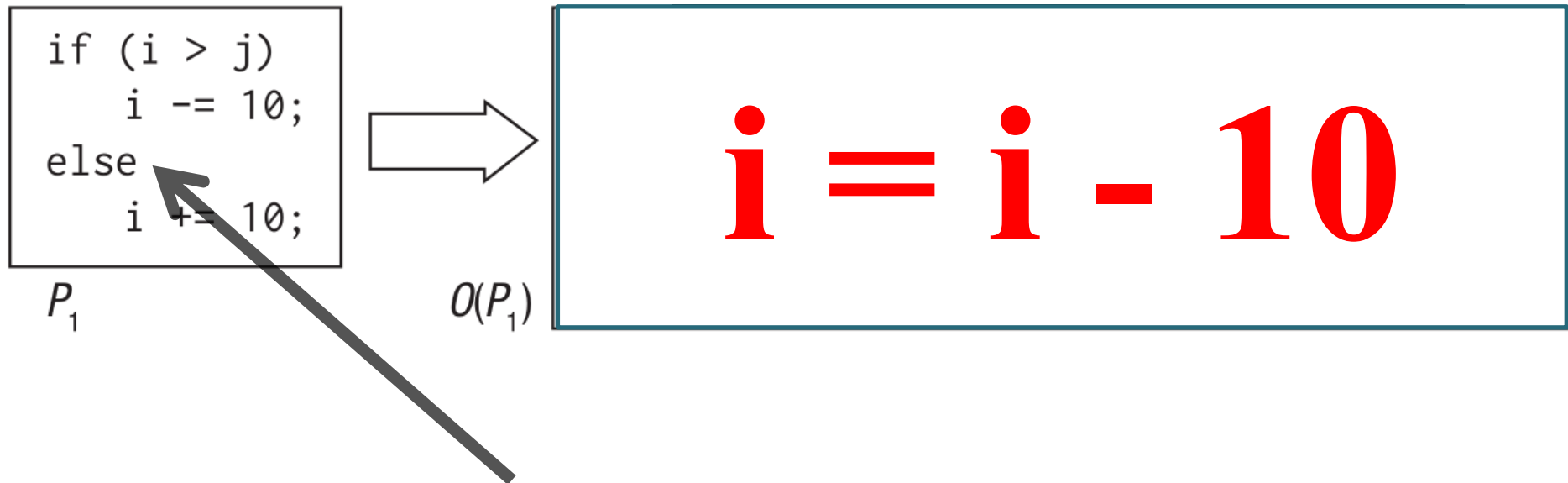


$O(P_2)$

$i = i + 10$

Detecting Obfuscator Bugs

Tigress



Conventional testing without MT can detect the failure **only when $i \leq j$ (hence not guaranteed)**

In comparison, MT guarantees detection of the failure in this example, because $O(P_1)$ and $O(P_2)$ always output different values for i .

```
if (i > j)
    i -= 10;
else
    i += 10;
```

P_1

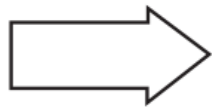


$O(P_1)$

$i = i - 10$

```
if (i <= j)
    i += 10;
else
    i -= 10;
```

P_2



$O(P_2)$

$i = i + 10$

Detecting Obfuscator Bugs

- Bugs/Issues also detected in all other obfuscators under test:
 - Cobfusc
 - open source Linux utility
 - Obfuscator-LLVM
 - open source tool in LLVM compilation suite
 - StunniX CXX-Obfus
 - commercial C/C++ obfuscator
 - users include many Fortune 500 companies

Related Work

- Detect compiler bugs, e.g.

Distinguished paper award, 35th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI '14):

Compiler Validation via Equivalence Modulo Inputs

Vu Le

Mehrdad Afshari

Zhendong Su

Department of Computer Science, University of California, Davis, USA

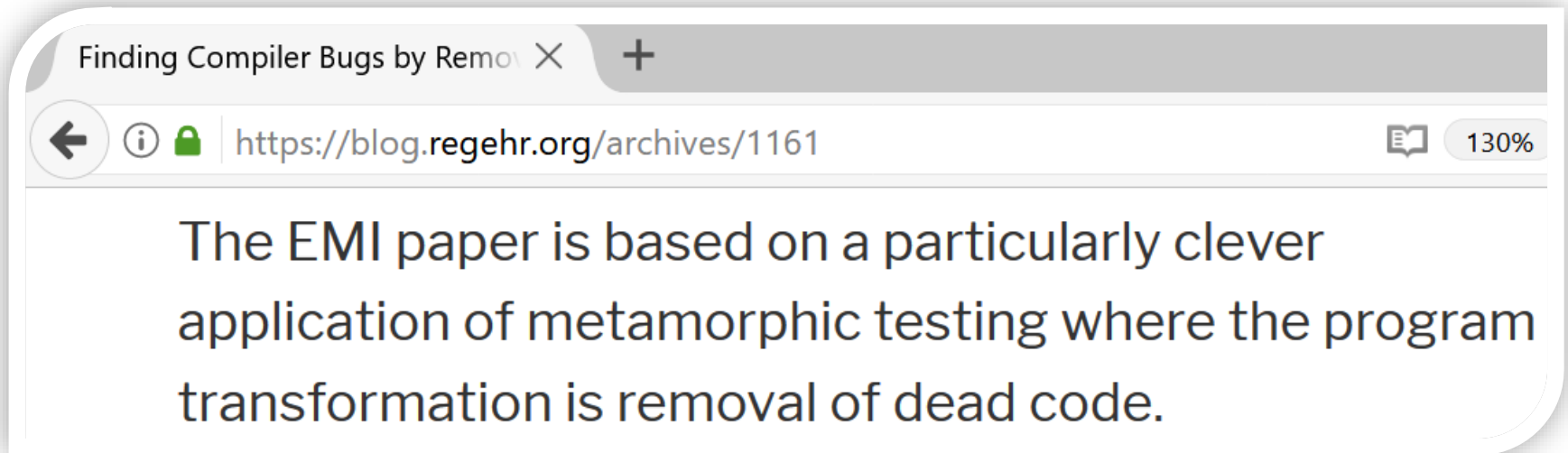
{vmle, mafshari, su}@ucdavis.edu

Abstract

We introduce *equivalence modulo inputs (EMI)*, a simple, widely applicable methodology for validating optimizing compilers. Our

Besides traditional manual code review and testing, the main compiler validation techniques include testing against popular validation suites (such as Plum Hall [21] and SuperTest [1]), verification [12, 13], translation validation [20, 22], and random test

Related Work



Compiler Validation via Equivalence Modulo Inputs

Vu Le

Mehrdad Afshari

Zhendong Su

Department of Computer Science, University of California, Davis, USA

{vmle, mafshari, su}@ucdavis.edu

Abstract

We introduce *equivalence modulo inputs (EMI)*, a simple, widely applicable methodology for validating optimizing compilers. Our

Besides traditional manual code review and testing, the main compiler validation techniques include testing against popular validation suites (such as Plum Hall [21] and SuperTest [1]), verification [12, 13], translation validation [20, 22], and random test

Related Work

- The MR: If source programs **p1.c** and **p2.c** are equivalent for input *I*, then their compiler-generated object programs **p1.exe** and **p2.exe** should also be equivalent with respect to *I*.

Compiler Validation via Equivalence Modulo Inputs

Vu Le

Mehrdad Afshari

Zhendong Su

Department of Computer Science, University of California, Davis, USA

{vmle, mafshari, su}@ucdavis.edu

Abstract

We introduce *equivalence modulo inputs (EMI)*, a simple, widely applicable methodology for validating optimizing compilers. Our

Besides traditional manual code review and testing, the main compiler validation techniques include testing against popular validation suites (such as Plum Hall [21] and SuperTest [1]), verification [12, 13], translation validation [20, 22], and random test

Related Work

- To construct **p2.c**:
 - execute **p1.exe** using arbitrary input I;
 - record code-coverage information with respect to **p1.c**;
 - create **p2.c** by randomly pruning some unexecuted statements from **p1.c**.

Compiler Validation via Equivalence Modulo Inputs

Vu Le

Mehrdad Afshari

Zhendong Su

Department of Computer Science, University of California, Davis, USA

{vmle, mafshari, su}@ucdavis.edu

Abstract

We introduce *equivalence modulo inputs (EMI)*, a simple, widely applicable methodology for validating optimizing compilers. Our

Besides traditional manual code review and testing, the main compiler validation techniques include testing against popular validation suites (such as Plum Hall [21] and SuperTest [1]), verification [12, 13], translation validation [20, 22], and random test

Related Work

- Report a compiler bug if **p2.exe(I)≠p1.exe(I)**.
- Researchers documented **147** confirmed, unique **bug** reports for the **GCC** and **LLVM** C compilers alone.
 - Last update (2019): **1634** bugs, **1076** fixed!

Compiler Validation via Equivalence Modulo Inputs

Vu Le

Mehrdad Afshari

Zhendong Su

Department of Computer Science, University of California, Davis, USA

{vmle, mafshari, su}@ucdavis.edu

Abstract

We introduce *equivalence modulo inputs (EMI)*, a simple, widely applicable methodology for validating optimizing compilers. Our

Besides traditional manual code review and testing, the main compiler validation techniques include testing against popular validation suites (such as Plum Hall [21] and SuperTest [1]), verification [12, 13], translation validation [20, 22], and random test

MT for Cybersecurity

- The Heartbleed “bug” was amongst the most widely known cybersecurity breaches
 - You should spend time studying what happened
- Read “MT for Cybersecurity”

